

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ  
РОССИЙСКОЙ ФЕДЕРАЦИИ  
федеральное государственное автономное образовательное учреждение  
высшего образования «Санкт-Петербургский политехнический  
университет Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 «Математика и компьютерные науки»

Отчет о выполнении курсовой работы  
по дисциплине «Функциональное программирование»  
Вариант №23.

Студент,  
группы 5130201/30101

\_\_\_\_\_ Мартыненко А. П.

Преподаватель

\_\_\_\_\_ Моторин Д.Е.

«\_\_\_\_\_» \_\_\_\_\_ 2025г.

Санкт-Петербург, 2025

# Содержание

|          |                                              |           |
|----------|----------------------------------------------|-----------|
| <b>1</b> | <b>Постановка задачи</b>                     | <b>3</b>  |
| <b>2</b> | <b>Особенности реализации</b>                | <b>6</b>  |
| 2.1      | Часть 1: игра «Лестница слов» . . . . .      | 6         |
| 2.1.1    | Типы данных . . . . .                        | 6         |
| 2.1.2    | Ассоциативный список . . . . .               | 7         |
| 2.1.3    | Ориентированный граф . . . . .               | 8         |
| 2.1.4    | Нахождение анаграмм . . . . .                | 11        |
| 2.1.5    | Построение графа слов . . . . .              | 13        |
| 2.2      | Часть 2: работа с N-граммами . . . . .       | 14        |
| 2.2.1    | Подготовка текста . . . . .                  | 14        |
| 2.2.2    | Построение N-граммной модели . . . . .       | 15        |
| 2.2.3    | Загрузка модели и генерация текста . . . . . | 16        |
| <b>3</b> | <b>Результаты работы</b>                     | <b>21</b> |
|          | <b>Выводы</b>                                | <b>23</b> |
|          | <b>Список использованных источников</b>      | <b>24</b> |

# 1 Постановка задачи

Цель: написание синтаксического анализатора и генератора текста по заданному слову или сочетанию слов.

Задачи:

1. Разработать синтаксический анализатор текстовых файлов. Функции:
  - 1.1. Считать текст из текстового файла (формат .txt), название которого вводит пользователь.
  - 1.2. Выполнить его анализ по следующим правилам: слова состоят только из букв, предложения состоят только из слов, предложения разделяются стандартными знаками препинания . ! ? ; : \n, , " ' - \_ ( ) [ ] { } < > . Поддерживаются тексты только на английском языке.
  - 1.3. Привести слова к нижнему регистру и записать предложения в словарь.
2. Составить модель N-грамм:
  - 2.1. Использовать N-граммы при  $N=4$ .
  - 2.2. N-грамма - это последовательность слов предложения при  $g = (w_1, w_2, \dots, w_n), 1 \leq n \leq N$ , где ключом является  $k = (w_1, w_2, \dots, w_a), 1 \leq a \leq N - 1$ , значением - продолжение предложения  $v = (w_{a+1}, w_{a+2}, \dots, w_{a+m}), m \leq N - a$ . При этом  $a + m \leq N$ . Значения N-граммы уникальны.
  - 2.3. Для каждого ключа формируется вероятность по формуле  $P = \frac{\text{count}(k,v)}{\sum_i \text{count}(k,v)}$ , где count - количество появлений N-граммы.
  - 2.4. По запросу пользователя сохранить модель в файл расширения .json в формате JSON.
3. Реализовать генерацию текстов:
  - 3.1. Пользователь задает начальное слово (одно, два или три).
  - 3.2. При вводе менее одного слова или более трех слов сообщить пользователю и повторить попытку ввода.
  - 3.3. По ключу выбрать значение в соответствии с правилами п. 2.2. и учетом вероятности п. 2.3. Длина сгенерированного сообщения случайна, но меньше, чем максимальная длина исходного текста.

- 3.4. При отсутствии слова в ключах словаря вывести сообщение об этом.
- 3.5. Вывести сгенерированное сообщение, которое начинается с заглавной буквы.
- 3.6. Запрос к ключам словаря не должен занимать более 2 секунд.
- 4. Организовать диалог двух N-граммных моделей:
  - 4.1. Модели созданы в соответствии с правилами п. 2.2. и 2.3.
  - 4.2. Модели созданы на двух разных текстах, написанных на английском языке. Каждый текст не короче 100000 слов.
  - 4.3. Взаимодействие реализуется через пользовательское взаимодействие. Пользователь вводит начальное слово и продолжительность диалога (количество сообщений).
  - 4.4. Сообщения чередуются: четные сообщения генерируются моделью 1, нечетные - моделью 2. Каждое сообщение начинается с большой буквы.
  - 4.5. Каждое новое сообщение основывается на последнем слове, паре или тройке из сообщения оппонента.
  - 4.6. Если ключ отсутствует среди ключей, описанных в п.4.5., модель перебирает все ключи, двигаясь от конца предложения к началу.
  - 4.7. В случае ненахождения ключей в п.4.6., модель генерирует случайное сообщение и помечает в начале восклицательным знаком.
- 5. Организовать консольное меню с командами:
  - 5.1. Создать модель N-грамм по заданному файлу.
  - 5.2. Сохранить словарь существующей модели.
  - 5.3. Сгенерировать предложение по вводимому слову.
  - 5.4. Запросить диалог двух моделей.
  - 5.5. Выйти из приложения.
- 6. Обработка ошибок выполняется:
  - 6.1. При выборе несуществующего пункта меню.
  - 6.2. При вводе слов, не подчиняющихся правилам п.1.2.

- 6.3. При работе с файлами: необходима возможность их открыть и считать данные.
- 6.4. При построении моделей.
- 6.5. При генерации текста.
- 6.6. При вводе слишком больших или пустых данных с клавиатуры или из файла.
- 6.7. При работе моделей в режиме диалога.

Тривиальный пример данных: «The cat sleeps. The cat eats meat.»

Выделенные N-граммы в формате ключ  $\rightarrow$  значение (частота):

the  $\rightarrow$  cat (2)  
the  $\rightarrow$  cat sleeps (1)  
the  $\rightarrow$  cat eats (1)  
the  $\rightarrow$  cat eats meat (1)  
the cat  $\rightarrow$  sleeps (1)  
the cat  $\rightarrow$  eats (1)  
the cat  $\rightarrow$  eats meat (1)  
the cat eats  $\rightarrow$  meat (1)  
cat  $\rightarrow$  sleeps (1)  
cat  $\rightarrow$  eats (1)  
cat  $\rightarrow$  eats meat (1)  
eats  $\rightarrow$  meat (1)

## 2 Особенности реализации

### 2.1 Часть 1: игра «Лестница слов»

Основная идея игры - построить цепочку слов от заданного начального слова к конечному по определенным правилам. Чтобы перейти от одного слова к другому, программа может:

1. Менять одну букву в слове.
2. Добавлять новую букву.
3. Удалять одну букву.
4. Произвольно менять порядок букв.
5. Каждое промежуточное слово цепочки должно существовать в словаре.

#### 2.1.1 Типы данных

Тип `AssocMap k v` предназначен для реализации ассоциативного списка, который содержит пары ключ-значение.

Листинг 1. Тип `AssocMap k v`

```
1 newtype AssocMap k v = AssocMap [(k, v)]
2 deriving (Show)
```

Тип `DiGraph` представляет орграф, созданный на основе хеш-таблицы.

Листинг 2. Тип `DiGraph a`

```
1 type DiGraph a = M.HashMap a [a]
```

Тип `SearchState` представляет состояние поиска в ширину. Тип `BiSearchState` представляет состояние двунаправленного поиска в ширину, состоящее из поиска от начальной вершины и от конечной вершины.

Листинг 3. Тип `BiSearchState`

```
1 type SearchState a = ([a], DiGraph a, DiGraph a)
2 type BiSearchState a = (SearchState a, SearchState a)
```

Тип `Dictionary` представляет словарь, загруженный из текстового файла.

Листинг 4. Тип `Dictionary`

```
1 type Dictionary = [BS.ByteString]
2 type PermutationMap = M.HashMap BS.ByteString [BS.ByteString]
```

Тип `PermutationMap` представляет карту анаграмм.

## 2.1.2 Ассоциативный список

Функция `empty` предназначена для создания пустого ассоциативного списка.

Листинг 5. Функция `empty`

```
1 empty :: AssocMap k v
2 empty = AssocMap []
```

Функция `elems` предназначена для получения только значений ассоциативного списка.

Листинг 6. Функция `elems`

```
1 elems :: AssocMap k v -> [v]
2 elems (AssocMap xs) = map snd xs
```

Функция `member` предназначена для проверки наличия ключа в ассоциативном списке. Рекурсивно для каждого элемента списка ключ сравнивается с текущим ключом списка, при совпадении функция возвращает `True`.

Листинг 7. Функция `member`

```
1 member :: Eq k => k -> AssocMap k v -> Bool
2 member key (AssocMap xs) = member' key xs
3 where
4 member' :: Eq k => k -> [(k, v)] -> Bool
5 member' _ [] = False
6 member' x ((x', _) : xs)
7 | x' == x = True
8 | otherwise = member' x xs
```

Функция `alter` предназначена для универсальной операции изменения значения по данному ключу. Если ключ не найден, функция получает `Nothing`, если найден - `Just value`. В зависимости от результата запись обновляется, удаляется или создается.

Листинг 8. Функция `alter`

```
1 alter :: Eq k => (Maybe v -> Maybe v) -> k -> AssocMap k v -> AssocMap k
  v
2 alter f key (AssocMap xs) = AssocMap (alter' f key xs)
3 where
4 alter' :: Eq k => (Maybe v -> Maybe v) -> k -> [(k, v)] -> [(k, v)]
5 alter' f key [] =
6 case f Nothing of
7 Nothing -> []
8 Just value -> [(key, value)]
9 alter' f key ((key', value') : xs)
10 | key == key' =
11 case f (Just value') of
12 Nothing -> xs
13 Just value -> (key, value) : xs
```

```

14 | otherwise =
15 (key', value') : alter' f key xs

```

Функция `delete` удаляет ключ из ассоциативного списка. Она реализована с помощью `alter`, который всегда возвращает `Nothing`, то есть удаляет пару из ассоциативного списка.

Функция `insert` добавляет новую пару или изменяет существующую. Она реализована с помощью `alter`, который всегда возвращает `Just value`, то есть обновляет или добавляет пару элементов.

Листинг 9. Функции `delete` и `insert`

```

1 delete :: Eq k => k -> AssocMap k v -> AssocMap k v
2 delete = alter (const Nothing)
3
4 insert :: Eq k => k -> v -> AssocMap k v -> AssocMap k v
5 insert key value = alter (const (Just value)) key

```

Функция `lookup` предназначена для поиска значений по ключу. Она рекурсивно проходит по списку и сравнивает текущий ключ с переданным в функцию. В случае, если совпадений не найдено, возвращает `Nothing`.

Листинг 10. Функция `lookup`

```

1 lookup :: Eq k => k -> AssocMap k v -> Maybe v
2 lookup key (AssocMap xs) = lookup' key xs
3 where
4 lookup' key [] = Nothing
5 lookup' key ((key', value) : xs)
6 | key == key' = Just value
7 | otherwise = lookup' key xs

```

Функция `findWithDefault` предназначена также для поиска значений по ключу, используя `lookup`.

Листинг 11. Функция `findwithDefault`

```

1 findWithDefault :: (Eq k) => v -> k -> AssocMap k v -> v
2 findWithDefault defaultValue key map =
3 case lookup key map of
4 Nothing -> defaultValue
5 Just value -> value

```

В отличие от `lookup`, данная функция возвращает переданный параметр по умолчанию в случае, если ключ не найден.

### 2.1.3 Ориентированный граф

Функция `empty` создает пустой орграф на основе хеш-таблицы (`Data.HashMap.Lazy`).



Листинг 12. Функция empty

```

1 empty :: DiGraph a
2 empty = M.empty

```

Функция addNode добавляет вершину в граф. Если она уже есть, то граф не изменяется. Если вершины нет, она создается с пустым списком соседей.

Листинг 13. Функция addNode

```

1 addNode :: (Hashable a, Eq a) => a -> DiGraph a -> DiGraph a
2 addNode =
3 M.alter
4 ( \mNodes ->
5 case mNodes of
6 Nothing -> Just []
7 value -> value
8 )

```

Функция addEdge добавляет ориентированное ребро между двумя вершинами. Если первая вершина отсутствует, она создается со списком, содержащим вторую. Иначе второй элемент добавляется в список соседей без дублирования.

Листинг 14. Функция addEdge

```

1 addEdge :: (Hashable a, Eq a) => (a, a) -> DiGraph a -> DiGraph a
2 addEdge (node, child) = M.alter insertEdge node
3 where
4 insertEdge Nothing = Just [child]
5 insertEdge (Just nodes) =
6 Just (L.nub (child : nodes))

```

Функция buildDiGraph предназначен для создания графа из списка пар вида (узел, потомок).

Листинг 15. Функция buildDiGraph

```

1 buildDiGraph :: (Hashable a, Eq a) => [(a, [a])] -> DiGraph a
2 buildDiGraph nodes = go nodes M.empty
3 where
4 go [] graph = graph
5 go ((key, value) : xs) graph = M.insert key value (go xs graph)

```

Функция children возвращает список всех потомков-детей указанной вершины. Если вершина отсутствует, по умолчанию возвращается пустой список.

Листинг 16. Функция children

```

1 children :: (Hashable a, Eq a) => a -> DiGraph a -> [a]
2 children = M.findWithDefault []

```

Функции deleteNode и deleteEdge удаляют вершину и ребро соответственно, если они существуют.

Листинг 17. Функция deleteNode и deleteEdge

```

1 deleteNode :: (Hashable a, Eq a) => a -> DiGraph a -> DiGraph a
2 deleteNode = M.delete
3
4 deleteEdge :: (Hashable a, Eq a) => (a, a) -> DiGraph a -> DiGraph a
5 deleteEdge (node, child) =
6   M.alter
7   ( \mNodes ->
8     case mNodes of
9       Nothing -> Just []
10      Just nodes ->
11        Just (L.delete child nodes)
12    )
13   node

```

Функция biBfsSearch предназначена для двунаправленного поиска в ширину по графу. Одновременно запускается поиск от начальной и конечной вершин. При пересечении фронтов поиска путь восстанавливается соединением двух частичных маршрутов.

Листинг 18. Функция biBfsSearch

```

1 biBfsSearch :: forall a. (Hashable a, Eq a) => DiGraph a -> a -> a ->
2   Maybe [a]
3 biBfsSearch graph start end
4 | start == end = Just [start]
5 | otherwise =
6   let fState = ([start], graph, empty)
7   bState = ([end], graph, empty)
8   in biBfsSearch' (fState, bState)
9   where
10    findSolution :: DiGraph a -> a -> [a]
11    findSolution g = go
12    where
13      go x =
14        case children x g of
15          [] -> [x]
16          (v : _) -> x : go v
17
18    checkOverlap :: BiSearchState a -> Maybe [a]
19    checkOverlap ((fFrontier', _, fPreds'), (bFrontier', _, bPreds')) =
20      let getSolution x =
21        let fPath = findSolution fPreds' x
22        bPath = findSolution bPreds' x
23        in L.reverse fPath ++ L.tail bPath
24      overlaps =
25        L.filter ('M.member' bPreds') fFrontier'
26      solutions =
27        L.sortOn L.length (L.map getSolution overlaps)
28      in if L.null solutions
29        then Nothing
30        else Just $ L.head solutions
31
32    checkSolution :: SearchState a -> a -> Maybe [a]
33    checkSolution (frontier, _, preds) node

```

```

33 | node 'L.elem' frontier = Just (findSolution preds node)
34 | otherwise = Nothing
35
36 biBfsSearch' :: BiSearchState a -> Maybe [a]
37 biBfsSearch' state@(fState@(fFrontier, _, _), bState@(bFrontier, _, _))
38 | L.null fFrontier && L.null bFrontier = Nothing
39 | isJust fSol = fmap L.reverse fSol
40 | isJust bSol = bSol
41 | isJust overlapSol = overlapSol
42 | otherwise = biBfsSearch' biState'
43 where
44 fSol = checkSolution fState end
45 bSol = checkSolution bState start
46 overlapSol = checkOverlap state
47 fState' = bfsSearchStep fState
48 bState' = bfsSearchStep bState
49 biState' = (fState', bState')
50
51 addMultiplePredecessors :: [(a, [a])] -> DiGraph a -> DiGraph a
52 addMultiplePredecessors [] g = g
53 addMultiplePredecessors ((n, ch) : xs) g =
54 addMultiplePredecessors xs (go n ch g)
55 where
56 go n [] g = g
57 go n (x : xs) g = go n xs (addEdge (x, n) g)
58
59 bfsSearchStep :: SearchState a -> SearchState a
60 bfsSearchStep (frontier, g, preds) =
61 let g' = deleteNodes frontier g
62     ch =
63     L.map
64     (\n -> (n, L.filter ('M.member' g') (children n g)))
65     frontier
66     frontier' = L.concatMap snd ch
67     preds' = addMultiplePredecessors ch preds
68 in (frontier', g', preds')

```

Вложенная функция `findSolution` реализует восстановление пути по графу предков, `checkOverlap` - наличие пересечения фронтов двух поисков, `checkSolution` - достижение целевой границы, `addMultiplePredecessors` - хранение предков конкретной вершины.

## 2.1.4 Нахождение анаграмм

Функция `empty` создает пустую карту анаграмм.

Листинг 19. Функция `empty`

```

1 empty :: PermutationMap
2 empty = M.empty

```

Функция `member` проверяет наличие ключа в структуре. Здесь и далее ключ - это отсортированное по буквам слово.

#### Листинг 20. Функция member

```
1 member :: BS.ByteString -> PermutationMap -> Bool
2 member key = M.member (BS.sort key)
```

Функция alter изменяет список слов, которые связаны с ключом.

#### Листинг 21. Функция alter

```
1 alter ::
2 ( Maybe [BS.ByteString] ->
3  Maybe [BS.ByteString]
4  ) ->
5  BS.ByteString ->
6  PermutationMap ->
7  PermutationMap
8  alter f key = M.alter f (BS.sort key)
```

Функция delete удаляет данные из структуры по ключу слова. Функция insert добавляет новое слово в список анаграмм. Функция lookup возвращает список всех слов, которые являются анаграммами переданного слова.

#### Листинг 22. Функции delete, insert, lookup

```
1 delete :: BS.ByteString -> PermutationMap -> PermutationMap
2 delete key = M.delete (BS.sort key)
3
4 insert :: BS.ByteString -> [BS.ByteString] -> PermutationMap ->
5   PermutationMap
6 insert key = M.insert (BS.sort key)
7
8 lookup :: BS.ByteString -> PermutationMap -> Maybe [BS.ByteString]
9 lookup key = M.lookup (BS.sort key)
```

Функция findWithDefault ищет значение по заданному ключу. Если ключ не найден - возвращается значение по умолчанию, иначе возвращается список анаграмм.

#### Листинг 23. Функция findWithDefault

```
1 findWithDefault ::
2 [BS.ByteString] ->
3 BS.ByteString ->
4 PermutationMap ->
5 [BS.ByteString]
6 findWithDefault defaultValue key map =
7 fromMaybe [] (PermutationMap.lookup key map)
```

Функция createPermutationMap строит карту анаграмм на основе списка слов.

#### Листинг 24. Функция createPermutationMap

```
1 createPermutationMap :: [BS.ByteString] -> PermutationMap
2 createPermutationMap = go empty
```

```

3 where
4 go permMap [] = permMap
5 go permMap (x : xs) = go (insertPermutation x permMap) xs
6 insertPermutation word = alter (insertList word) word
7 insertList word Nothing = Just [word]
8 insertList word (Just words) = Just $ word : words

```

Для каждого слова вычисляется ключ, который является отсортированным по буквам слово, и по ключу слово помещаются в соответствующую группу.

### 2.1.5 Построение графа слов

Функция `readDictionary` предназначена для чтения словаря из файла. Содержимое файла разбивается на строки и фильтруется так, что остаются только буквы латинского алфавита.

Листинг 25. Функция `readDictionary`

```

1 readDictionary :: FilePath -> IO Dictionary
2 readDictionary filepath = do
3   dictionaryContent <- C.readFile filepath
4   let lines = C.lines dictionaryContent
5   words = L.map (C.filter ('L.elem' ['a' .. 'z'])) lines
6   return words

```

Функция `delete` удаляет первое вхождение указанной буквы из слова.

Листинг 26. Функция `delete`

```

1 delete :: Char -> BS.ByteString -> BS.ByteString
2 delete ch string = case C.uncons string of
3   Just (x, xs) -> if ch == x then xs else C.cons x (delete ch xs)
4   Nothing -> C.empty

```

Функция `computeCandidates` предназначена для вычисления всех возможных слов, которые можно получить из исходного с помощью правил игры «Лестница слов». Каждое созданное слово проверяется на существование ключа, связанные слова включаются в итоговый список. Из результата исключается исходное слово.

Листинг 27. Функция `computeCandidates`

```

1 computeCandidates :: PM.PermutationMap -> BS.ByteString -> [BS.
   ByteString]
2 computeCandidates map word =
3   let candidates = modified ++ removed ++ added ++ [word]
4   perms =
5     L.concatMap
6     (\x -> PM.findWithDefault [] x map)
7     candidates
8   in L.filter (/= word) (L.nub perms)
9   where

```

```

10 added = [C.cons x word | x <- ['a' .. 'z']]
11 removed = [delete x word | x <- C.unpack word]
12 modified =
13 [C.cons x (delete y word) | x <- ['a' .. 'z'], y <- C.unpack word, x /=
    y]

```

Функция `mkLadderGraph` предназначена для построения орграфа, где вершины - это слова словаря, а ребра ведут ко всем его «потомкам», построенным по правилам игры.

Листинг 28. Функция `mkLadderGraph`

```

1 mkLadderGraph :: Dictionary -> G.DiGraph BS.ByteString
2 mkLadderGraph dict = G.buildDiGraph nodes
3 where
4   map = PM.createPermutationMap dict
5   nodes =
6   L.map (\w -> (w, computeCandidates map w)) dict
7
8 ladderSolve :: Dictionary -> String -> String -> Maybe [BS.ByteString]
9 ladderSolve dict start end =
10 let g = mkLadderGraph dict
11 in G.biBfsSearch g (C.pack start) (C.pack end)

```

Функция `ladderSolve` предназначена для нахождения кратчайшего пути между словами, если он существует. Для этого она строит граф слов и запускает двудirectional поиск в ширину.

## 2.2 Часть 2: работа с N-граммами

### 2.2.1 Подготовка текста

Функция `normalizeText` предназначена для приведения текста к виду для парсинга: весь текст находится в нижнем регистре, удалены лишние пробелы и символы.

Листинг 29. Функция `normalizeText`

```

1 normalizeText :: Text -> Text
2 normalizeText txt =
3 T.unwords . T.words . T.toLower $
4 T.map replaceBad txt
5 where
6 replaceBad c | c `elem` ['\n', '\t'] = '.'
7 | isDigit c = ' '
8 | otherwise = c

```

Функция `splitSentences` разделяет текст по символам завершения предложения, каждое полученное предложение делит по пробелам на слова. Функция оставляет только непустые предложения.

Листинг 30. Функция splitSentences

```

1 splitSentences :: Text -> [[Text]]
2 splitSentences =
3   filter (not . null) .
4   map (filter (not . T.null) . T.split ('elem' wordSeparators)) .
5   T.split ('elem' sentenceSeparators)
6 where
7   sentenceSeparators = [ '.', '!', '?', ';', ':', '\n', ',', '"', '\'', '‘’',
8     , '-', '_', '(', ')', '[', ']', '{', '}', '<', '>' ]
9   wordSeparators = [ ' ' ]

```

## 2.2.2 Построение N-граммной модели

Тип Key хранит ключи (1-3 слова), тип Value хранит возможное продолжение (1-3 слова), тип KVCounts хранит частоты встречаемости. Тип NGramModel предназначен для хранения структуры N-граммы, где order - максимальная длина ключа, counts - словарь N-грамм и их частот.

Листинг 31. Тип данных

```

1 type Key = [Text]
2 type Value = [Text]
3 type KVCounts = M.Map Key (M.Map Value Int)
4 data NGramModel = NGramModel
5 { order :: Int
6   , counts :: KVCounts
7 }

```

Функция nextTokens предназначена для генерации случайного предложения с учетом вероятностей. Она находит все варианты значений и выбирает пропорционально их частотам.

Листинг 32. Функция nextTokens

```

1 nextTokens :: NGramModel -> [Text] -> IO (Maybe [Text])
2 nextTokens model key = do
3   let mp = M.lookup key (counts model)
4   case mp of
5     Nothing -> pure Nothing
6     Just vals -> chooseP vals
7   where
8     chooseP :: M.Map Value Int -> IO (Maybe [Text])
9     chooseP m
10    | M.null m = pure Nothing
11    | otherwise = do
12      let total = sum (M.elems m)
13      randomValue <- randomRIO (0, total - 1)
14      let result = findWithP (M.toList m) randomValue 0
15      pure $ Just result
16   findWithP :: [(Value, Int)] -> Int -> Int -> Value
17   findWithP [] _ _ = error "Should not happen"
18   findWithP ((v, count):rest) target current
19   | target < current + count = v

```

```
20 | otherwise = findWithP rest target (current + count)
```

Функция `getProbabilities` преобразует частоты встречаемости продолжений ключа в вероятности.

Листинг 33. Функция `getProbabilities`

```
1 getProbabilities :: NGramModel -> [Text] -> [(Value, Double)]
2 getProbabilities model key =
3 case M.lookup key (counts model) of
4 Nothing -> []
5 Just vals ->
6 let total = fromIntegral $ sum (M.elems vals)
7 in map \(v, c) -> (v, fromIntegral c / total) (M.toList vals)
```

Функция `buildModel` строит N-граммную модель из списка предложений. Она вызывает вложенную функцию `buildFromSentence` для каждого предложения и объединяет результаты предложений с суммированием частот.

Листинг 34. Функция `buildModel`

```
1 buildModel :: Int -> [[Text]] -> NGramModel
2 buildModel maxOrder sentences =
3 NGramModel maxOrder $ M.unionsWith (M.unionWith (+)) (map
4   buildFromSentence sentences)
5 where
6 buildFromSentence :: [Text] -> KVCounts
7 buildFromSentence tokens = foldr insertKV M.empty allKVs
8 where
9 allKVs = concatMap (ngramsFrom tokens) [1..3]
10 ngramsFrom :: [Text] -> Int -> [(Key, Value)]
11 ngramsFrom xs k =
12 [ (take k rest, take vLen (drop k rest))
13   | rest <- tails xs
14     , length rest >= k + 1
15     , vLen <- [1..(4 - k)]
16     , length rest >= k + vLen
17 ]
18 insertKV :: (Key, Value) -> KVCounts -> KVCounts
19 insertKV (k,v) = M.insertWith (M.unionWith (+)) k (M.singleton v 1)
```

Функция `buildFromSentence` получает список всех ключей и значений предложения через `ngramsFrom` и добавляет их в общий массив с помощью `insertKV`. Функция `ngramsFrom` строит все N-граммы длиной до 4.

### 2.2.3 Загрузка модели и генерация текста

Функция `createModel` предназначена для создания N-граммной модели на основе текстового файла. Она работает в двух режимах: путь к файлу заранее передан или путь к файлу вводится пользователем. В



случае успешного чтения файла функция передает текст в парсер, находит максимальную длину предложения и строит модель с помощью `buildModel`.

Листинг 35. Функция `createModel`

```
1 createModel :: Maybe FilePath -> IO (NGramModel, Int)
2 createModel mFile = do
3   filePath <- getValidFilePath mFile
4   let n = 4
5   corpus <- T.readFile filePath
6   let normalized = normalizeText corpus
7   sentences = splitSentences normalized
8   maxLen = maximum (map length sentences)
9   putStrLn $ filePath ++ " loaded. Sentences: " ++ show (length sentences)
10    ++ ", max length: " ++ show maxLen
11   let model = buildModel n sentences
12   putStrLn "Model built successfully!"
13   return (model, maxLen)
```

Функция `saveModel` предназначена для сохранения N-граммной модели в выбранный пользователем файл в формате JSON.

Листинг 36. Функция `saveModel`

```
1 saveModel :: NGramModel -> IO ()
2 saveModel model = do
3   putStrLn "Enter filename to save model dictionary (must be .json file):"
4   filename <- getLine
5   case validateJsonFilename filename of
6     Left errorMsg -> do
7       putStrLn $ "Error: " ++ errorMsg
8       saveModel model
9     Right validFilename -> do
10      BL.writeFile validFilename (A.encode model)
11      putStrLn $ "Model saved to " ++ validFilename
12
13 instance ToJSON NGramModel where
14   toJSON (NGramModel ord cnts) =
15     object
16     [ "order" .= ord
17     , "counts" .= M.toList (M.map M.toList cnts)
18     ]
```

Функция `generateFromContext` строит новое предложение, начиная с ключа. Функция берет текущий ключ, с помощью `nextTokens` получает продолжение, создает новый ключ из последних 1-3 слов и продолжает подобный цикл, пока предложение не достигнет максимальной длины.

Листинг 37. Функция `generateFromContext`

```
1 generateFromContext :: NGramModel -> [T.Text] -> Int -> IO [T.Text]
2 generateFromContext model start maxLen = do
3   go start start
4   where
```

```

5 go currentKey acc
6 | length acc >= maxLen = do
7 pure acc
8 | otherwise = do
9 nextWords <- nextTokens model currentKey
10 case nextWords of
11 Nothing -> do
12 pure acc
13 Just newWords -> do
14 let newAcc = acc ++ newWords
15 let newKey = take (order model) (reverse (take 3 (reverse newAcc)))
16 go newKey newAcc

```

Функция `generateContinuation` получает от пользователя начальные 1-3 слова, проверяет корректность ввода и вызывает `generateFromContext`.

Листинг 38. Функция `generateContinuation`

```

1 generateContinuation :: NGramModel -> Int -> IO ()
2 generateContinuation model maxLen = do
3   putStrLn "Enter 1-3 starting words:"
4   line <- getLine
5
6   case validateInput line of
7   Left errorMsg -> do
8     putStrLn $ "Error: " ++ errorMsg
9     generateContinuation model maxLen
10  Right ws -> do
11    myLen <- randomRIO (1, maxLen)
12    result <- generateFromContext model ws maxLen
13    let capitalizedResult = capitalizeSentence result
14    putStrLn $ "\nGenerated text:\n" ++ T.unpack (T.unwords
      capitalizedResult)

```

Функция `generateRandomSentence` перебирает все ключи модели и обрабатывает ситуацию, когда текущего ключа нет в ключах модели. В таком случае она выбирает случайный ключ и с помощью `generateFromContext` генерирует последовательность слов.

Листинг 39. Функция `generateRandomSentence`

```

1 generateRandomSentence :: NGramModel -> Int -> IO [T.Text]
2 generateRandomSentence model maxLen = do
3   let allKeys = M.keys (counts model)
4   if null allKeys
5   then return ["I", "have", "nothing", "to", "say"]
6   else do
7     randomIndex <- randomRIO (0, length allKeys - 1)
8     let randomKey = allKeys !! randomIndex
9     generateFromContext model randomKey maxLen

```

Функция `startDialog` запускает диалог двух N-граммных моделей. Она загружает модели из файлов с заранее прописанными текстами, спрашивает у пользователя начальный ключ, количество реплик и запускает цикл диалога `dialogLoop`.

Листинг 40. Функция startDialog

```

1 startDialog :: IO ()
2 startDialog = do
3   prideModel <- createModel (Just "Pride.txt")
4   mobyModel <- createModel (Just "mobydick.txt")
5   startingWords <- getValidStartingWords
6   turns <- getValidTurns
7   putStrLn "\nDialog between Elizabeth (Pride and Prejudice) and Captain
      Ahab (Moby Dick)"
8   let (prideModelNG, prideMaxLen) = prideModel
9   (mobyModelNG, mobyMaxLen) = mobyModel
10  dialogLoop prideModelNG prideMaxLen mobyModelNG mobyMaxLen startingWords
      turns 0

```

Функция dialogLoop позволяет организовать обмен сообщениями между двумя моделями. Если текущая реплика четная - пишет первая модель, нечетная - пишет вторая модель. Для написания предложения функция вызывает generateResponse и передает последние 1-3 слова следующей модели.

Листинг 41. Функция dialogLoop

```

1 dialogLoop :: NGramModel -> Int -> NGramModel -> Int -> [T.Text] ->
   Int -> Int -> IO ()
2 dialogLoop model1 maxLen1 model2 maxLen2 lastWords turns currentTurn
3 | currentTurn >= turns = do
4   putStrLn "\nEnd of the dialog"
5   return ()
6 | otherwise = do
7   let speaker = if even currentTurn then "Elizabeth" else "Captain Ahab"
8   let currentModel = if even currentTurn then model1 else model2
9   let currentMaxLen = if even currentTurn then maxLen1 else maxLen2
10  response <- generateResponse currentModel currentMaxLen lastWords
11  let finalResponse = response
12  putStrLn $ speaker ++ ": " ++ T.unpack (T.unwords finalResponse)
13  dialogLoop model1 maxLen1 model2 maxLen2 response turns (currentTurn +
      1)

```

Функция generatePossibleKeys создает набор возможных ключей.

Листинг 42. Функция generatePossibleKeys

```

1 generatePossibleKeys :: [T.Text] -> Int -> [[T.Text]]
2 generatePossibleKeys context _modelOrder =
3   map (:[]) (reverse context)

```

Функция findResponseForKeys пытается найти подходящий ключ среди всех возможных. Если ключ найден, то она вызывает generateFromContext, иначе - generateRandomSentence.

Листинг 43. Функция findResponseForKeys

```

1 findResponseForKeys :: NGramModel -> [[T.Text]] -> Int -> IO [T.Text]
2 findResponseForKeys model [] maxLen = do
3   randomSentence <- generateRandomSentence model maxLen

```

```

4 return $ capitalizeSentence ("!" : randomSentence)
5 findResponseForKeys model (key:keys) maxLen = do
6 let probs = getProbabilities model key
7 if null probs
8 then findResponseForKeys model keys maxLen
9 else do
10 sentence <- generateFromContext model key maxLen
11 return $ capitalizeSentence sentence

```

Функция `generateResponse` предназначена для выбора корректного ключа по последним словам предыдущей фразы.

Листинг 44. Функция `generateResponse`

```

1 generateResponse :: NGramModel -> Int -> [T.Text] -> IO [T.Text]
2 generateResponse model maxLen context = do
3 let possibleKeys = generatePossibleKeys context (order model)
4 response <- findResponseForKeys model possibleKeys maxLen
5 return response

```

Она создает список вариантов с помощью `generatePossibleKeys`, передает список в `findResponseForKeys` и возвращает сгенерированную фразу.

### 3 Результаты работы

На рисунке 1 показано меню для начала работы. Обработка неверного ввода представлена на рисунке 2.

```
N-gramm app:
1 - create N-gram model from file
2 - save model dictionary to file
3 - generate sentence continuation
4 - dialog between two models
5 - exit
Enter your choice (1-5):
```

Рис. 1. Консольное меню

```
Enter your choice (1-5):
df
Invalid choice! Please enter a number from 1 to 5.
```

Рис. 2. Обработка неверного ввода

На рисунке 3 показана загрузка текста из файла в словарь. На рисунке 4 пользователю предлагается ввести название файла, куда сохраняется словарь.

```
Enter your choice (1-5):
1
Enter path to corpus file (e.g., corpus.txt):
test.txt
test.txt loaded. Sentences: 25, max length: 8
Model built successfully!
```

Рис. 3. Загрузка текста

```
Enter your choice (1-5):
2
Enter filename to save model dictionary:
dict.txt
Model saved to dict.txt
```

Рис. 4. Сохранение словаря

На рисунке 5 показана генерация продолжения предложения по введенному слову пользователя.

```
Enter your choice (1-5):
3
Enter 1-3 starting words:
sing

Generated text:
Sing happy songs in the trees
```

Рис. 5. Генерация предложения

На рисунке 6 представлен результат генерации диалога между двумя моделями. Случай, когда предложение генерируется случайно из-за отсутствия введенного ключа, показан на рисунке 7.

```
Enter starting word or phrase:
happy
Enter number of dialog's sentences:
6

Dialog between Elizabeth (Pride and Prejudice) and Captain Ahab (Moby Dick)
Elizabeth: Happy to oblige you
Captain Ahab: You would not catch me in a brown study
Elizabeth: Study of my life to avoid those weaknesses which often expose a strong understanding to ridicule
Captain Ahab: To find one stub
Elizabeth: One whose knowledge of a sister
Captain Ahab: Sister of charity did this charitable aunt charity

End of the dialog
```

Рис. 6. Генерация диалога

```
Enter starting word or phrase:
fgfg
Enter number of dialog's sentences:
4

Dialog between Elizabeth (Pride and Prejudice) and Captain Ahab (Moby Dick)
Elizabeth: ! Amid very complaisant smiles and general encouragement
Captain Ahab: General thing
Elizabeth: Thing was too recent for gaiety
Captain Ahab: For captain sleet to describe

End of the dialog
```

Рис. 7. Отсутствующий ключ в словаре

Тестирование подтвердило, что приложение работает корректно, обрабатывая ситуации наличия и отсутствия ключей в словаре, а также некорректный ввод пользователя.

## Выводы

В ходе работы успешно реализован синтаксический анализатор и генератор текста по заданному слову или сочетанию слов. Анализатор разработан на модели N-грамм, где  $N=4$ . Написаны функции загрузки текста из файла, сохранения словаря в файл, генерация предложения по введенному слову на основе созданной модели и генерация диалога между двумя моделями на определенных текстах. Взаимодействие с программой реализовано с помощью пользовательского взаимодействия. Тестирование программы показало, что все функции работают корректно и обрабатывают частные случаи и некорректное поведение пользователя.

## Список литературы

- [1] Курт У. «Программируй на Haskell». М.: Издательство, 2019. 320 стр.
- [2] Липовача М. «Изучай Haskell во имя добра». М.: ДМК Пресс, 2012. 490 стр.
- [3] Р. Hagenlocher «Learn Haskell by example». Manning Publications Co, 2025. 1127 стр.
- [4] Learn Haskell by example // github : [сайт]. URL: <https://github.com/phagenlocher/learn-haskell-by-example/tree/main/chapter5%2B6> (дата обращения: 11.11.2025).